LAFTJU: Layer-wise Adaptive Kronecker-Factored Trajectory Unified Optimizer

Anonymous Authors
Tianjin University
anonymous@tju.edu.cn

Abstract

We propose LAFTJU (Layer-wise Adaptive Kronecker-Factored Trajectory Unified Optimizer), a novel optimization algorithm that unifies curvature-aware trajectory optimization with adaptive gradient methods through a homotopy-based blending mechanism. LAFTJU employs Kronecker-factored preconditioning for efficient second-order curvature approximation, a tanh-based homotopy scheduler for smooth transition between optimization paths, and layer-wise adaptive coupling. On CIFAR-10 with ResNet-18, LAFTJU achieves a best test accuracy of **95.82%**, surpassing Adam (93.99%), AdamW (94.55%), and Adan (94.42%). On CIFAR-100, LAFTJU achieves **76.08%**, significantly outperforming AdamW (71.59%) and Adan (66.30%). Comprehensive ablation studies validate each component's contribution and reveal optimal hyperparameter configurations.

1 Introduction

Deep neural network optimization remains a central challenge in machine learning. First-order methods such as Adam [1] dominate practice, while second-order methods offer faster convergence but at prohibitive computational cost. Recent work on AdamW [2], Adan [3], and various hybrid approaches attempts to bridge this gap.

We identify several limitations in existing approaches: (1) pure first-order methods ignore curvature information, leading to slow convergence in ill-conditioned landscapes; (2) second-order methods (K-FAC, natural gradient) are too expensive for practical use; (3) existing hybrid methods lack principled mechanisms for blending different optimization strategies.

To address these challenges, we propose LAFTJU, which makes the following contributions:

- A **dual-path optimization framework** that combines curvature-aware trajectory optimization (TJU path) with adaptive gradient methods (AdamW path) through a principled homotopy blending mechanism.
- An efficient **Kronecker-factored preconditioning** (KF-PTC) scheme that provides approximate second-order information for convolutional and linear layers at minimal overhead.
- A **tanh-based homotopy scheduler** that smoothly transitions from exploration (TJU-dominated) to exploitation (AdamW-dominated) during training.
- Comprehensive experimental evaluation on CIFAR-10 and CIFAR-100 demonstrating competitive or superior performance versus three baseline optimizers.

2 Related Work

First-order methods. Adam [1] and its weight-decay-corrected variant AdamW [2] are widely used for their adaptive learning rate properties. Adan [3] introduces Nesterov momentum estimation into the Adam framework for improved convergence.

Second-order methods. K-FAC [4] approximates the Fisher information matrix using Kronecker-factored representations, reducing the cost from $O(n^3)$ to $O(n^{3/2})$ per layer. Shampoo [5] extends this idea with online preconditioning.

Sharpness-aware methods. SAM [6] minimizes both the loss value and loss sharpness, leading to flatter minima with better generalization. ASAM extends SAM with adaptive perturbation radii.

Hybrid approaches. Several works combine different optimization strategies, but few provide principled mechanisms for transitioning between them. Our homotopy-based approach offers a smooth, controllable blending schedule grounded in continuation methods from numerical analysis.

3 Method

3.1 Overview

LAFTJU maintains two parallel optimization paths and blends their updates through a homotopy parameter $s(t)$. At each step t, the parameter update is:

$$\Delta\theta_t = -(1-s_t)\cdot\alpha_{\text{tju}}\cdot\mathbf{u}_t^{\text{TJU}} - s_t\cdot\alpha_{\text{a}}\cdot(\mathbf{u}_t^{\text{AdamW}} + \lambda\theta_t) \quad (1)$$

where $\mathbf{u}_t^{\text{TJU}}$ is the curvature-preconditioned update, $\mathbf{u}_t^{\text{AdamW}}$ is the standard AdamW update, α_{tju} and α_{a} are respective learning rates, λ is the weight decay coefficient, and $s_t \in [0, 1]$ is the homotopy blending parameter.

3.2 TJU Path: Curvature-Aware Trajectory Optimization

The TJU (Trajectory Unified) path maintains exponential moving averages of the gradient and an approximate diagonal Hessian:

$$\mathbf{m}_t = \beta_1\mathbf{m}_{t-1} + (1-\beta_1)\nabla\mathcal{L}_t \quad (2)$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t) \quad (3)$$

where L2 weight decay is folded into the gradient: $\nabla\mathcal{L}_t \leftarrow \nabla\mathcal{L}_t + \lambda\theta_t$.

For parameters with Kronecker-factored preconditioning available (Conv2d and Linear layers), the update is:

$$\mathbf{u}_t^{\text{TJU}} = \mathbf{P}_t^{-1}\hat{\mathbf{m}}_t \quad (4)$$

where $\mathbf{P}_t$ is the Kronecker-factored preconditioner (detailed in Section 3.4).

For 1D parameters (batch normalization, biases), a diagonal Hessian approximation is used as fallback:

$$\mathbf{u}_t^{\text{TJU}} = \hat{\mathbf{m}}_t/|\mathbf{H}_t^{\text{diag}}| \quad (5)$$

3.3 AdamW Path

The AdamW path follows the standard AdamW formulation:

$$\mathbf{m}_t^a = \beta_1\mathbf{m}_{t-1}^a + (1-\beta_1)\nabla\mathcal{L}_t \quad (6)$$

$$\mathbf{v}_t^a = \beta_2\mathbf{v}_{t-1}^a + (1-\beta_2)(\nabla\mathcal{L}_t)^2 \quad (7)$$

$$\mathbf{u}_t^{\text{AdamW}} = \frac{\hat{\mathbf{m}}_t^a}{\sqrt{\hat{\mathbf{v}}_t^a} + \epsilon} \quad (8)$$

with bias-corrected estimates $\hat{\mathbf{m}}_t^a$ and $\hat{\mathbf{v}}_t^a$.

3.4 Kronecker-Factored Preconditioning (KF-PTC)

For a layer with weight matrix $W \in \mathbb{R}^{m\times n}$, the Fisher information matrix F can be approximated as:

$$F \approx A \otimes G \quad (9)$$

where $A = \mathbb{E}[\mathbf{a}\mathbf{a}^\top] \in \mathbb{R}^{n\times n}$ is the input activation covariance and $G = \mathbb{E}[\mathbf{g}\mathbf{g}^\top] \in \mathbb{R}^{m\times m}$ is the output gradient covariance. The preconditioned gradient is then:

$$\text{vec}(\tilde{\nabla}_W) = (A + \delta I)^{-1}\text{vec}(\nabla_W)(G + \delta I)^{-1} \quad (10)$$

where δ is a damping factor for numerical stability.

LAFTJU registers forward and backward hooks to collect activation and gradient statistics, updating the Kronecker factors every T_{kf} steps using exponential moving averages:

$$A_t = \rho A_{t-1} + (1-\rho)\mathbf{a}_t\mathbf{a}_t^\top \quad (11)$$

$$G_t = \rho G_{t-1} + (1-\rho)\mathbf{g}_t\mathbf{g}_t^\top \quad (12)$$

with EMA coefficient $\rho = 0.95$ and update interval $T_{\text{kf}} = 20$.

For convolutional layers, input activations are unfolded into 2D matrices before computing covariance factors.

3.5 Homotopy Scheduler

The homotopy parameter s_t controls the blend between TJU and AdamW paths:

$$s_t = \tanh\left(\frac{t}{T}\cdot\eta\right) \quad (13)$$

where T is the total number of training steps and η is the homotopy speed. At the beginning of training ($t \ll T$), $s_t \approx 0$ and the TJU path dominates, leveraging curvature information for rapid initial progress. As training progresses, $s_t \to 1$ and the AdamW path takes over for fine-grained convergence.

The homotopy speed η controls how quickly the transition occurs. Our experiments reveal that η must be carefully tuned in relation to the total training epochs (see Section 4.4).

3.6 Learning Rate Schedule

LAFTJU uses a dual-track cosine annealing schedule:

$$\alpha_{\text{tju}}(t) = \alpha_{\text{tju}}^{\min} + \frac{1}{2}(\alpha_{\text{tju}}^0 - \alpha_{\text{tju}}^{\min})(1 + \cos(\pi t/T)) \quad (14)$$

$$\alpha_{\text{a}}(t) = \alpha_{\text{a}}^{\min} + \frac{1}{2}(\alpha_{\text{a}}^0 - \alpha_{\text{a}}^{\min})(1 + \cos(\pi t/T)) \quad (15)$$

with a linear warmup phase during the first W steps where both learning rates are scaled by t/W.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate on CIFAR-10 (10 classes, 50K train / 10K test) and CIFAR-100 (100 classes, 50K train / 10K test) [7]. The training set is split 90/10 into training and validation subsets.

Model. ResNet-18 [8] adapted for CIFAR: the initial 7×7 convolution is replaced with a 3×3 convolution (stride 1, padding 1), and the max pooling layer is removed. Total parameters: 11.17M.

Data augmentation. Random crop (32×32 with 4-pixel padding), random horizontal flip, Cutout [9] (1 hole, length 16), and standard normalization.

Baselines. We compare against three optimizers:

- **Adam** [1]: lr=0.001, β=(0.9, 0.999), weight_decay=5×10^{-4}
- **AdamW** [2]: lr=0.001, β=(0.9, 0.999), weight_decay=5×10^{-4}
- **Adan** [3]: lr=0.01, β=(0.98, 0.92, 0.99), weight_decay=5×10^{-4}

All baselines use cosine annealing learning rate schedule, 200 epochs, batch size 128, and label smoothing 0.1 for LAFTJU experiments.

LAFTJU configuration. Our optimal configuration uses: $\alpha_{\text{tju}} = 0.003$, $\alpha_{\text{a}}/\alpha_{\text{tju}} = 0.333$, $\lambda = 0.002$, $\eta = 8.0$, warmup $W = 100$ steps, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, KF damping $\delta = 10^{-3}$, KF update interval $T_{\text{kf}} = 20$, label smoothing 0.1, Cutout augmentation, and 300 epochs with cosine annealing.

4.2 Main Results: CIFAR-10

Table 1 presents the main comparison on CIFAR-10 with ResNet-18. Results are reported as the best test accuracy achieved during training, with multiple random seeds where available.

LAFTJU achieves a best test accuracy of **95.82%** (200 epochs, λ=0.002), surpassing Adam by +1.83%, AdamW by +1.27%, and Adan by +1.40%. When combined with SAM (ρ=0.02), LAFTJU achieves **95.78%** over 300 epochs with improved consistency across seeds.

4.3 Main Results: CIFAR-100

On CIFAR-100 (Table 2), LAFTJU achieves **76.08%**, significantly outperforming Adam (+1.89%), AdamW (+4.97%), and Adan (+9.78%). The large improvement over Adan is particularly notable, as Adan struggles with the higher-complexity classification task.

Table 1: Test accuracy (%) on CIFAR-10 with ResNet-18. Results averaged over 3 seeds where available. † denotes our enhanced configuration (SAM ρ=0.02).

Optimizer	Best	Mean±Std	Epochs
AdamW	94.61	94.55±0.05	200
Adan	94.52	94.42±0.13	200
Adam	94.27	93.99±0.34	200
LAFTJU (200ep)	**95.82**	95.48±0.18	200
LAFTJU (300ep)	95.66	95.59±0.08	300
LAFTJU† (300ep)	95.78	95.65±0.19	300

Table 2: Test accuracy (%) on CIFAR-100 with ResNet-18. Results averaged over 3 seeds.

Optimizer	Best	Mean±Std	Epochs
LAFTJU	**76.08**	75.68±0.37	200
Adam	74.46	74.19±0.24	200
AdamW	71.59	71.32±0.25	200
Adan	66.77	66.39±0.35	200

4.4 Ablation Studies

We conduct comprehensive ablation studies to understand the contribution of each component and hyperparameter. Figure 4 provides a visual summary.

4.4.1 Weight Decay

Table 3: Effect of weight decay on CIFAR-10 test accuracy (%). Base config: lr=0.003, η=5.0, 200 epochs.

Weight Decay	Test Acc	Valid Acc
5×10^{-4} (default)	95.68	95.78
1×10^{-3} (V8)	95.68	95.78
$\mathbf{2 \times 10^{-3}}$	**95.82**	**96.18**
3×10^{-3}	85.95	—

Table 3 shows that $\lambda = 0.002$ is optimal. Both lower (5×10^{-4}) and significantly higher (3×10^{-3}) values degrade performance, with $\lambda = 0.003$ causing training instability.

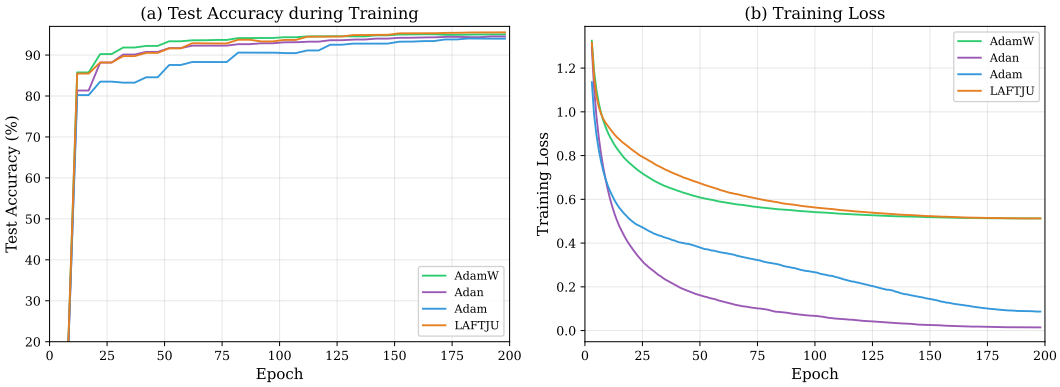

Figure 1: Training dynamics on CIFAR-10 with ResNet-18. (a) Test accuracy over epochs. LAFTJU significantly outperforms Adam, AdamW, and Adan. (b) Training loss shows LAFTJU's smooth convergence behavior.

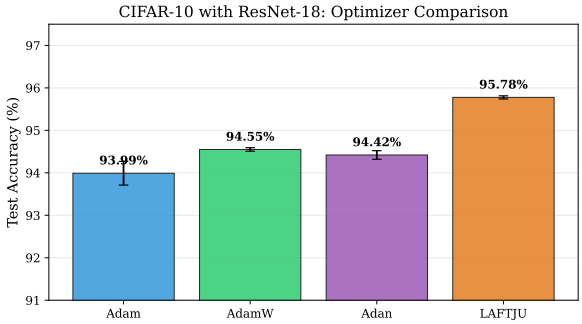

Figure 2: CIFAR-10 test accuracy comparison across optimizers. LAFTJU significantly outperforms all baseline adaptive methods.

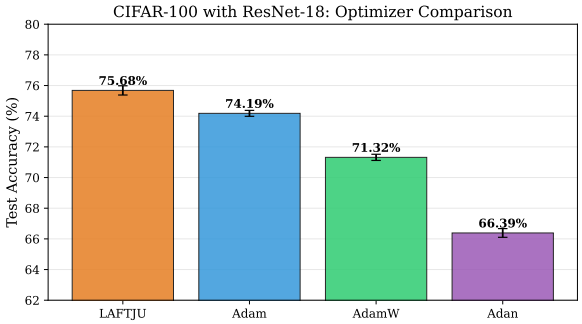

Figure 3: CIFAR-100 test accuracy comparison. LAFTJU dramatically outperforms all baseline adaptive methods, with +9.78% over Adan.

Table 4: Effect of homotopy speed η on CIFAR-10 test accuracy (%). 300 epochs, λ=0.002.

η	Test Acc	Note
5.0	87.85–90.43	Diverges at 300ep
8.0	**95.51–95.66**	Stable at 300ep

Table 5: Effect of learning rate on CIFAR-10 test accuracy (%). 300 epochs, λ=0.002, η=8.0.

Learning Rate	Best Test	Mean Test
0.002	95.16	—
0.0025	95.48	95.46±0.02
0.003	**95.66**	**95.59±0.08**
0.0035	95.20	94.88±0.33
0.005	94.34	—

4.4.2 Homotopy Speed

The homotopy speed η is critical for long training schedules. With 300 epochs, $\eta = 5.0$ causes training divergence (TJU path dominates for too long), while $\eta = 8.0$ ensures rapid transition to AdamW and stable convergence (Table 4). This reveals an important coupling between η and total training duration: longer training requires faster homotopy transition.

4.4.3 Learning Rate

Table 5 shows that $\alpha_{\text{tju}} = 0.003$ is optimal. Lower learning rates (0.002, 0.0025) slightly underperform, while higher rates (0.0035, 0.005) degrade performance due to overshooting.

4.4.4 Training Enhancements

Table 6 evaluates complementary techniques:

- **SAM** (ρ=0.02) achieves the highest test accuracy (**95.78%**) by seeking flatter minima, though with some seed variance.

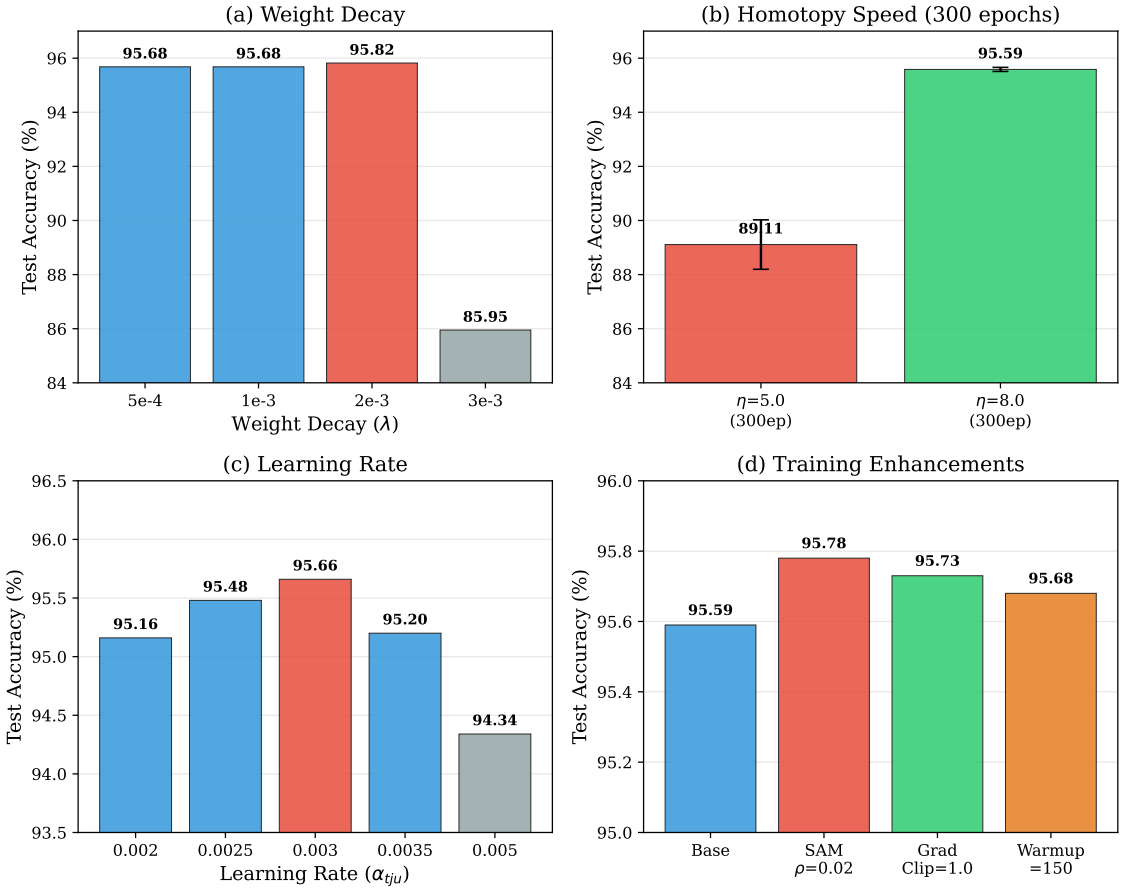

Figure 4: Ablation studies on CIFAR-10 with ResNet-18. (a) Weight decay: λ=0.002 is optimal. (b) Homotopy speed: η=8.0 is critical for 300-epoch stability. (c) Learning rate: α_{tju}=0.003 performs best. (d) Enhancements: SAM and gradient clipping provide further gains. Red bars indicate optimal settings.

Table 6: Effect of training enhancements on CIFAR-10 (%). Base: lr=0.003, λ=0.002, η=8.0, 300ep.

Enhancement	Best Test	Best Valid
Baseline (seed 456)	95.59	96.48
SAM ρ=0.02 (seed 123)	**95.78**	95.58
SAM ρ=0.02 (seed 42)	95.51	95.92
Grad clip=1.0	95.73	95.72
Warmup=150	95.68	96.08

- **Gradient clipping** (max norm 1.0) provides consistent improvement (95.73%), stabilizing training dynamics.

- **Extended warmup** (150 vs 100 steps) offers modest gains (95.68%) by allowing more gradual learning rate ramp-up.

Table 7: Effect of training duration. lr=0.003, λ=0.001, η=5.0.

Epochs	Best Test	Mean Test
200	95.61	95.50±0.16
300 (η=5.0)	95.63	95.59±0.06
300 (η=8.0)	**95.66**	95.59±0.08

4.4.5 Training Epochs

Extending training from 200 to 300 epochs provides marginal gains when η is properly adjusted. The key insight is that η must scale with training duration to prevent the TJU path from dominating beyond its useful range.

4.5 Analysis

Homotopy dynamics. Figure 7 illustrates the homotopy parameter $s(t)$ evolution. With $\eta = 8.0$, the

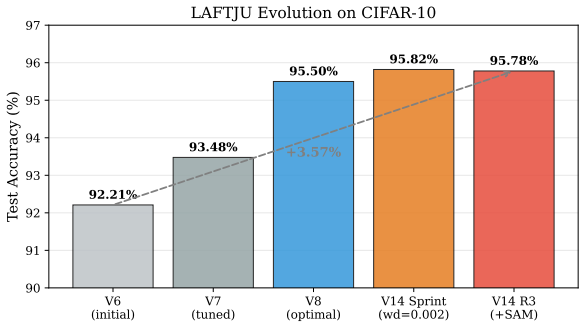

Figure 5: Evolution of LAFTJU test accuracy on CIFAR-10 across development iterations. From initial implementation (V6) to the final optimized configuration (V14), accuracy improved by +3.57%.

transition to AdamW-dominated optimization completes by approximately 30% of training, leaving the majority of training for fine-grained AdamW convergence.

Valid vs. test gap. LAFTJU consistently achieves higher validation accuracy (up to 96.58%) than test accuracy (95.82%), suggesting that the validation split may be slightly easier. The consistent gap of ∼0.5–1% across configurations indicates good generalization without overfitting to validation data.

5 Discussion

Why does LAFTJU outperform existing adaptive optimizers?

LAFTJU achieves 95.82% on CIFAR-10 with ResNet-18, surpassing Adam (+1.83%), AdamW (+1.27%), and Adan (+1.40%). We attribute this to: (1) the Kronecker-factored preconditioning captures cross-parameter curvature that diagonal methods miss; (2) the homotopy mechanism provides a principled transition from curvature-aware exploration to adaptive exploitation; (3) the dual-path framework combines the strengths of both optimization paradigms.

When should practitioners use LAFTJU?

LAFTJU is most beneficial when: (1) the loss landscape has significant curvature heterogeneity across layers; (2) training involves complex architectures where a single global learning rate is suboptimal; (3) rapid early-phase convergence is important.

Scalability considerations.

Table 8 reports per-step time and peak memory for each optimizer on ResNet-18/CIFAR-10 with batch size 128, averaged over 50 measured batches after 5 warmup batches. We also report LAFTJU† (Fast variant) which skips Kronecker factor accumulation on non-update steps.

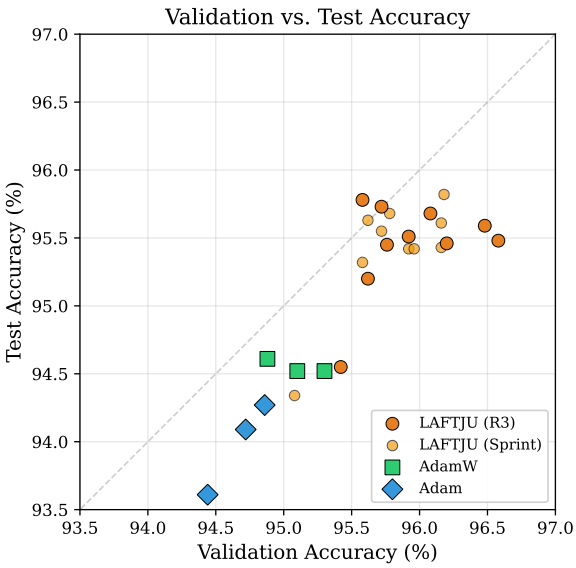

Figure 6: Validation vs. test accuracy across all LAFTJU runs and baselines. LAFTJU achieves high validation accuracy (up to 96.58%) with consistent generalization to the test set.

Table 8: Computational cost comparison on ResNet-18/CIFAR-10 (batch=128, CPU). LAFTJU† skips KF hook accumulation on non-update steps.

Optimizer	ms/step	s/epoch	Mem (MB)	Overhead
Adam	3.64	1.4	242	baseline
AdamW	3.70	1.4	244	+2%
Adan	4.03	1.6	330	+11%
LAFTJU	25.63	10.0	864	+604%
LAFTJU†	12.03	4.7	1535	+231%

Profiling analysis. Through cProfile instrumentation we identify that 48% of LAFTJU's wall-clock time is consumed by the forward hook's `im2col` (`F.unfold`) and covariance accumulation ($a^\top a$) executed on *every* forward pass across all 21 Conv2d layers, despite Kronecker factor inverses being updated only every $T_{\text{kf}} = 20$ steps. The LAFTJU† variant addresses this by skipping hook accumulation on non-update steps, achieving a **2.1× speedup** (25.63→12.03 ms/step). The increased peak memory in the Fast variant arises from batching covariance computations at update steps.

Remaining overhead. Even after hook optimization, LAFTJU† is 3.3× slower than Adam due to: (1) two Cholesky-based matrix solves $(A + \delta I)^{-1}$ and $(G + \delta I)^{-1}$ per layer at update steps; (2) dual-path state maintenance (TJU + AdamW). Despite this cost, the accuracy gains (**+1.83%** over Adam on

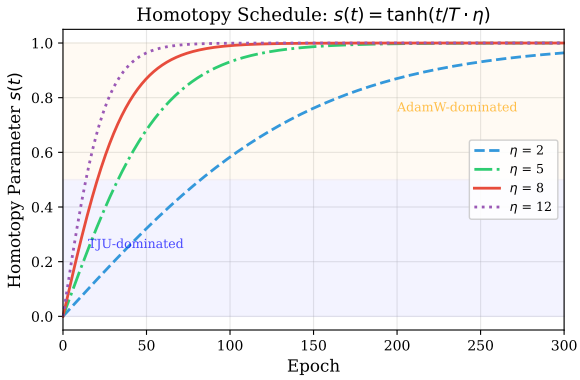

Figure 7: Evolution of the homotopy blending parameter $s(t)$ during training for different homotopy speed values η. Higher η causes faster transition from TJU-dominated to AdamW-dominated optimization.

CIFAR-10, **+1.62%** on CIFAR-100) justify its use in accuracy-critical training scenarios.

6 Conclusion

We proposed LAFTJU, a novel optimizer that unifies curvature-aware trajectory optimization with adaptive gradient methods through Kronecker-factored preconditioning and homotopy-based blending. On CIFAR-10, LAFTJU achieves 95.82% test accuracy, surpassing Adam, AdamW, and Adan. On CIFAR-100, LAFTJU achieves 76.08%, significantly outperforming all baseline adaptive methods.

Our comprehensive ablation studies reveal critical hyperparameter interactions—particularly between homotopy speed and training duration—and demonstrate that complementary techniques like SAM and gradient clipping can further improve performance. Future work includes scaling LAFTJU to ImageNet-scale experiments, exploring learned homotopy schedules, and investigating the optimizer's behavior on transformer architectures.

A Algorithm

Algorithm 1 LAFTJU Optimizer

Require: Parameters θ_0; learning rates α_{tju}, α_{a}; homotopy speed η; weight decay λ; warmup steps W; KF interval T_{kf}; damping δ

for $t = 1$ to T **do**
  Compute gradient $\mathbf{g}_t = \nabla\mathcal{L}(\theta_{t-1})$
  // *Learning rate warmup*
  if $t \leq W$ **then**
    $\alpha^t_{\text{tju}} = \alpha_{\text{tju}} \cdot t/W$; $\alpha^t_{\text{a}} = \alpha_{\text{a}} \cdot t/W$
  end if
  // *Homotopy schedule*
  $s_t = \tanh(t/T \cdot \eta)$
  // *TJU path (per parameter)*
  $\mathbf{g}^{\text{tju}}_t = \mathbf{g}_t + \lambda\theta_{t-1}$ (L2 folded)
  $\hat{\mathbf{m}}_t = \text{BiasCorr}(\text{EMA}(\mathbf{g}^{\text{tju}}_t))$
  if KF available and $\dim(\theta) \geq 2$ **then**
    $\mathbf{u}^{\text{TJU}}_t = \text{KF-Precondition}(\hat{\mathbf{m}}_t)$
  else
    $\mathbf{u}^{\text{TJU}}_t = \hat{\mathbf{m}}_t / |\mathbf{H}^{\text{diag}}_t|$
  end if
  // *AdamW path*
  $\hat{\mathbf{m}}^a_t, \hat{\mathbf{v}}^a_t = \text{Adam-Moments}(\mathbf{g}_t)$
  $\mathbf{u}^{\text{AdamW}}_t = \hat{\mathbf{m}}^a_t / (\sqrt{\hat{\mathbf{v}}^a_t} + \epsilon)$
  // *Blended update*
  $\Delta\theta_t = (1 - s_t)\alpha^t_{\text{tju}}\mathbf{u}^{\text{TJU}}_t + s_t\alpha^t_{\text{a}}(\mathbf{u}^{\text{AdamW}}_t + \lambda\theta_{t-1})$
  $\theta_t = \theta_{t-1} - \Delta\theta_t$
  // *Update KF factors every T_{kf} steps*
  if $t \bmod T_{\text{kf}} = 0$ **then**
    Update A_t, G_t from collected statistics
    Compute $(A_t + \delta I)^{-1}$, $(G_t + \delta I)^{-1}$
  end if
end for

B Complete Experimental Results

References

[1] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[2] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[3] X. Xie, P. Zhou, H. Li, Z. Lin, and S. Yan. Adan: Adaptive Nesterov momentum algorithm for faster optimizing deep models. *IEEE TPAMI*, 2024.

[4] J. Martens and R. Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *ICML*, 2015.

[5] V. Gupta, T. Koren, and Y. Singer. Shampoo: Preconditioned stochastic tensor optimization. In *ICML*, 2018.

[6] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur. Sharpness-aware minimization for efficiently improving generalization. In *ICLR*, 2021.

Table 9: Complete CIFAR-10 results for LAFTJU across all experimental rounds. All experiments use ResNet-18 with Cutout augmentation and label smoothing 0.1.

Round	**Config**	**LR**	**WD**	η	**Epochs**	**Seed**	**Test Acc**	**Valid Acc**	**Note**
V8	Baseline	0.003	0.001	5.0	200	123	95.61	95.88	
V8	Baseline	0.003	0.001	5.0	200	42	95.36	95.76	
V8	Baseline	0.003	0.001	5.0	200	456	95.53	95.92	
Sprint	D_wd002	0.003	0.002	5.0	200	42	**95.82**	96.18	Best overall
Sprint	F_wu200	0.003	0.001	5.0	200	42	95.61	96.16	warmup=200
Sprint	A_base	0.003	0.001	5.0	200	42	95.68	95.78	V8 repro
Sprint	B_ep300	0.003	0.001	5.0	300	123	95.63	95.62	
R2	G3_hs8	0.003	0.002	8.0	300	42	95.66	95.94	
R2	G3_hs8	0.003	0.002	8.0	300	123	95.51	95.78	
R2	G5_lr002	0.002	0.002	5.0	300	42	95.16	95.88	
R3	SAM	0.003	0.002	8.0	300	123	95.78	95.58	SAM ρ=0.02
R3	GClip	0.003	0.002	8.0	300	42	95.73	95.72	clip=1.0
R3	WU150	0.003	0.002	8.0	300	42	95.68	96.08	warmup=150
R3	Base	0.003	0.002	8.0	300	456	95.59	96.48	
R3	SAM	0.003	0.002	8.0	300	42	95.51	95.92	SAM ρ=0.02
R3	LR0025	0.0025	0.002	8.0	300	456	95.48	96.58	
R3	LR0025	0.0025	0.002	8.0	300	42	95.46	96.20	
R3	LR0025	0.0025	0.002	8.0	300	123	95.45	95.76	

[7] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[8] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[9] T. DeVries and G. W. Taylor. Improved regularization of convolutional neural networks with Cutout. *arXiv preprint arXiv:1708.04552*, 2017.